

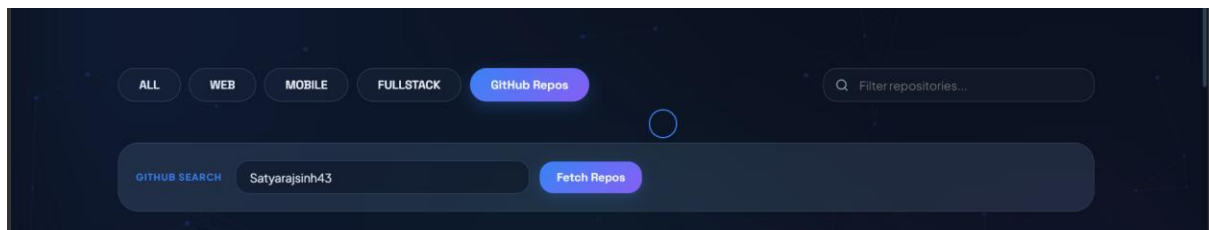
PRACTICAL-3

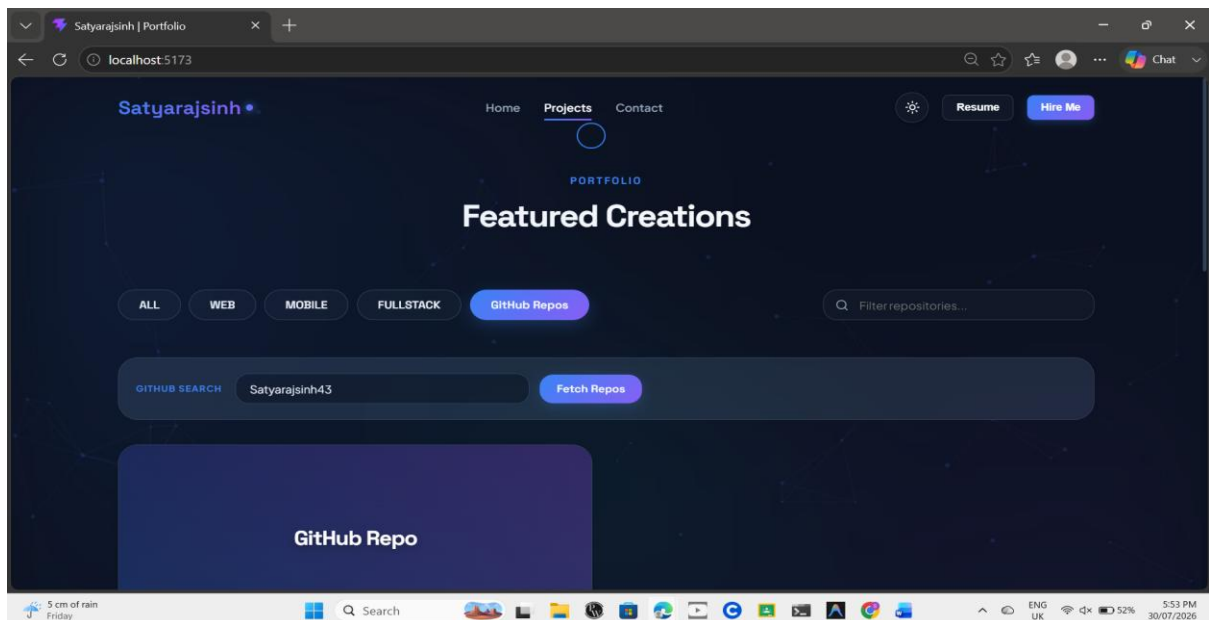
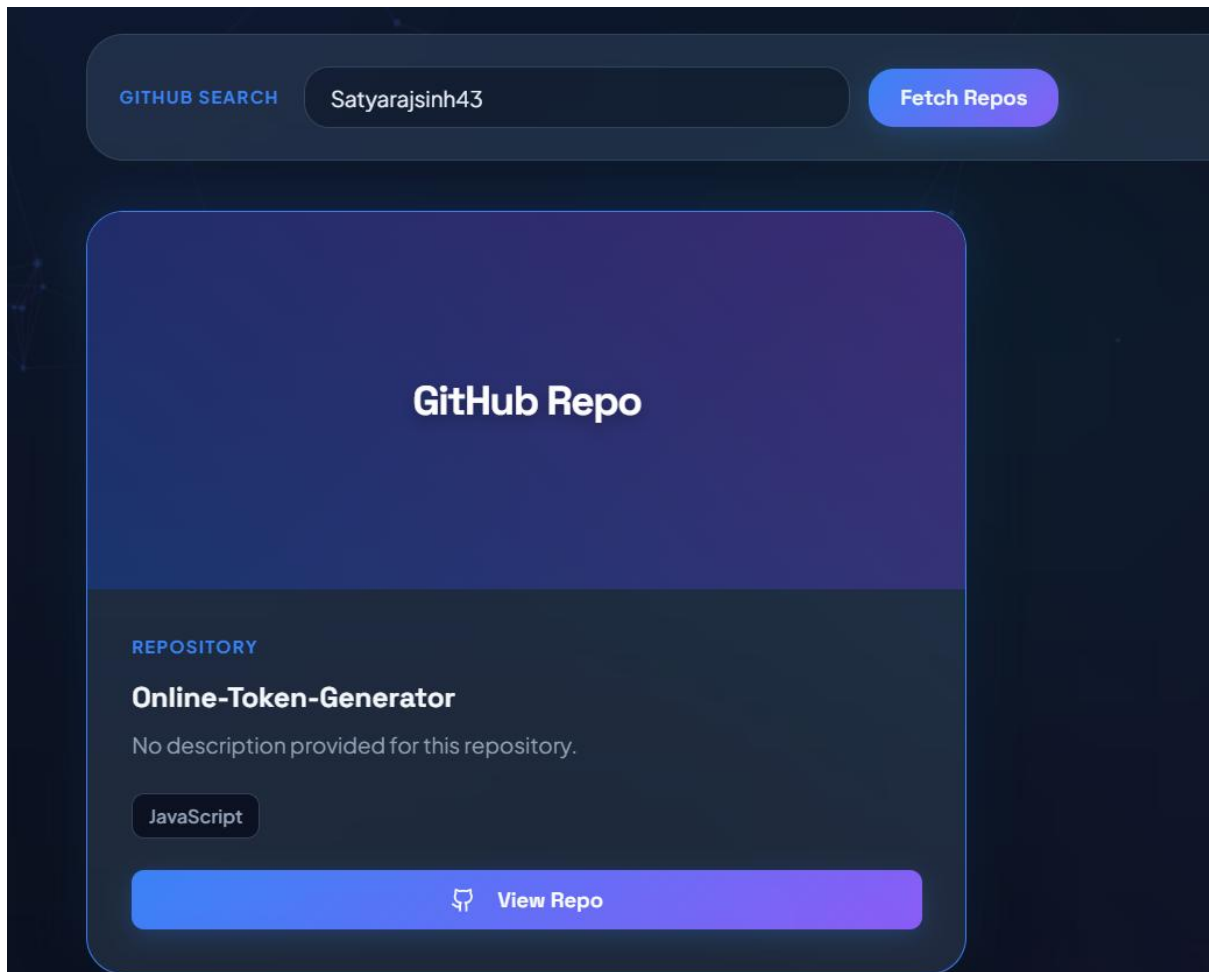
Problem Defination:

- Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component shown while the request is in progress.
- Implement an error message component shown if the API call fails.
- Render at least the repository name and URL for each item returned.

GitHub: https://github.com/24dit061-Satyarajsinh/24DIT061_ITUE301-AWD

OUTPUT:





Key Questions:-

1. Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?

useEffect() runs after the component is rendered and is ideal for API calls. If fetch() is called directly inside the component body, it executes on every render, causing repeated API requests. Using useEffect() with an empty dependency array ([]) ensures the data is fetched only once when the component mounts.

2. What is the difference between a loading state and an error state, and why must both be handled separately?

A loading state indicates that the API request is still in progress, while an error state indicates that the request has failed. Handling both separately provides clear feedback to the user. This improves the application's usability and user experience.

3. How would the user experience change if loading and error states were not implemented?

Without a loading state, users may think the application has frozen while waiting for data. Without an error state, users will not know why the data failed to load.

Implementing both states makes the application more responsive, informative, and user-friendly.

Conclusion

This practical demonstrated how to integrate a public REST API into a React application using useEffect and fetch. It also highlighted the importance of handling loading and error states to provide a better user experience. Dynamic data fetching makes the application more interactive, reliable, and responsive.